

Email : steinerro@student.ethz.ch

Unterlagen :
(Polybox)



Plan heute

- Mini Quiz
- Ex. Sheet 9
- Graphenproblem: kürzester Weg
 - BFS
 - Dijkstra

Mini Quiz :

10min

Passwort :

shortestpath3

Gruppenwechsel

Wichtiges aus der Vorlesung

- BFS
- Dijkstra

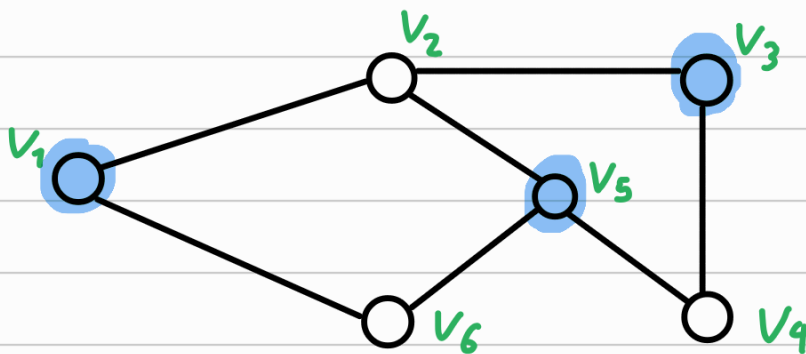
Exercise Sheet 9

9.4

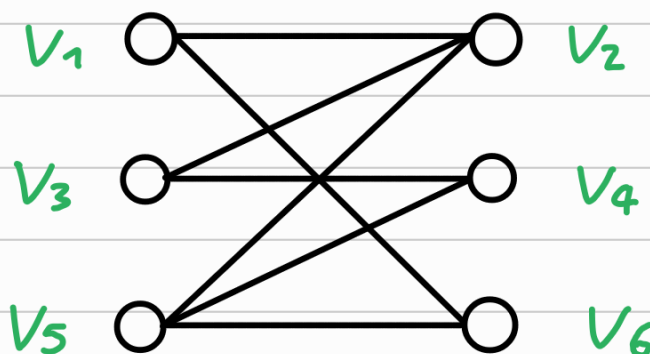
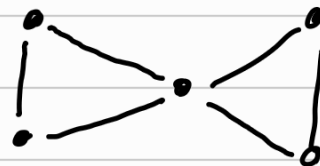
Definition 1. We say that a graph G is *Eulerian* if it contains a closed Eulerian Walk (Eulerzyklus).

Definition 2. A graph $G = (V, E)$ is *bipartite* if it is possible to partition the vertices in two sets V_1 and V_2 (i.e. $V_1 \cap V_2 = \emptyset$ and $V_1 \cup V_2 = V$) such that every edge $\{u, v\} \in E$ has one endpoint in V_1 and the other in V_2 .

Bipartite Graphen:



Andere Darstellung



Theorem 1. A graph is bipartite if and only if it does not contain any cycle of odd length.

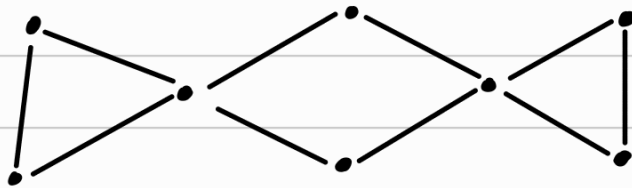
$\Leftrightarrow$

(1) Every graph G that is bipartite and Eulerian must have an even number of edges.

G ist bipartit $\Rightarrow G$ hat keinen Kreis ungerader Länge (Th. 1)

$\Rightarrow$ Kein Zyklus von G hat ungerade Länge.

(Vergleich Algorithmus Eulerzyklus finden, aus Teilzyklen/kreisen)



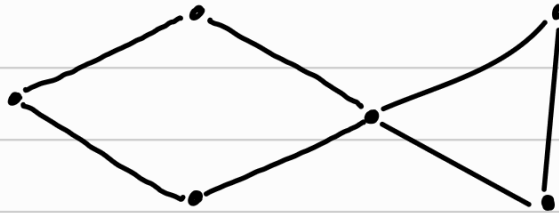
G hat Eulerzyklus $\Rightarrow$ Alle Kanten von G bilden Zyklus

$\Rightarrow |E|$ ist gerade.

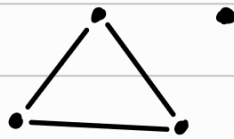
(Intuitive Ansicht.
Master Solution ist präziser)

(2) Every Eulerian graph G that has an even number of vertices must also have an even number of edges.

Lösung 1: Ein nicht bipartiter, Euler-Graph mit gerader Anzahl Knoten



Lösung 2: ein nicht verbundener Graph



Theorem 1. A graph is bipartite if and only if it does not contain any cycle of odd length.



Beweis (a)

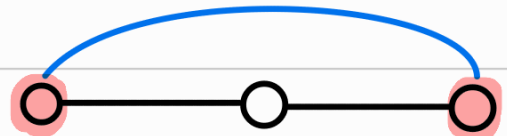
" $\Rightarrow$ " : Sei A ein bipartiter Graph

Annahme : A enthält Kreis C mit ungerader Länge

jeder zweite Knoten in C muss die gleiche Farbe haben (A ist bipartit)

$\Rightarrow$ Erster und letzter Knoten in C haben gleiche Farbe.

$\Rightarrow$ A ist nicht Bipartit.



Ein Widerspruch. Damit ist die Annahme falsch.

" $\Leftarrow$ " Sei A ein Graph ohne Kreise ungerader Länge

sei $v \in V$ beliebig

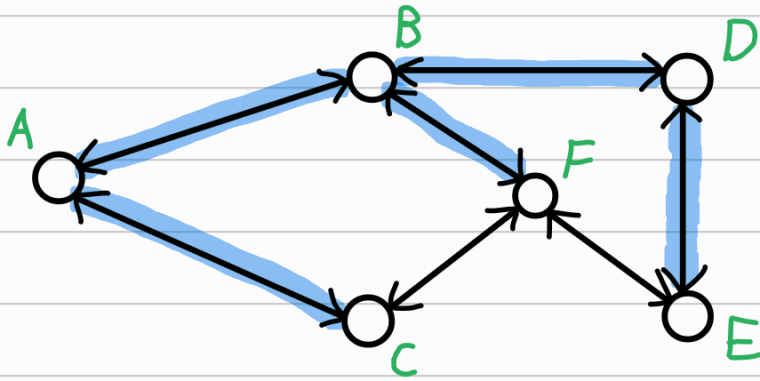
$V_1 = \{u \in V \mid \text{min. Distanz von } v \text{ zu } u \text{ ist } \underline{\text{ungerade}}\}$

$V_2 = \{u \in V \mid \text{min. Distanz von } v \text{ zu } u \text{ ist } \underline{\text{gerade}}\}$

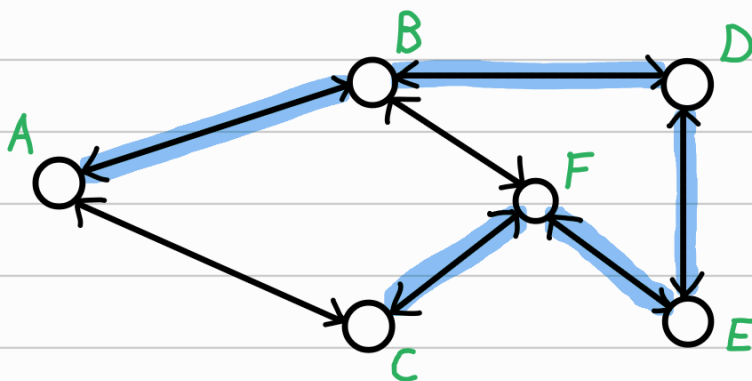
V_1, V_2 bilden eine Bipartition

Begründung: BFS (Breadth first search)

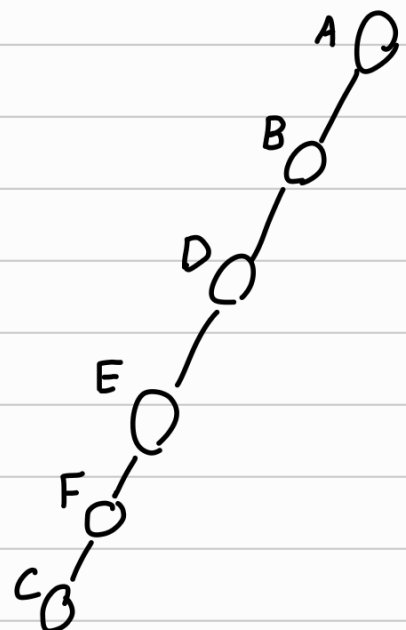
Graphen Traversierung (wie DFS)



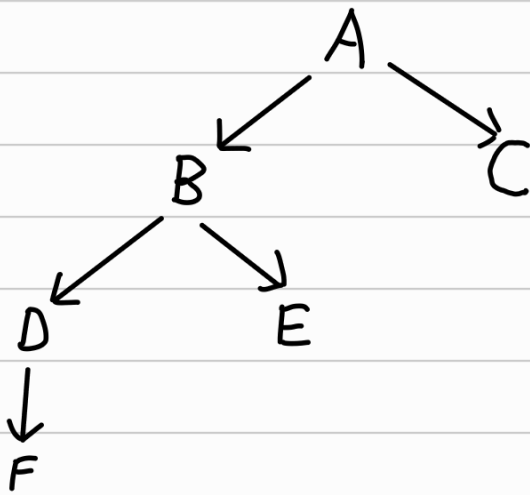
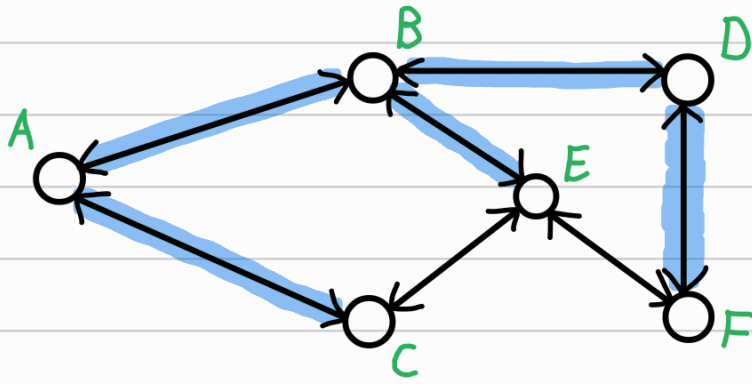
Vergleich zu DFS



Tiefensuchbaum



BFS Baum / Distanzbaum



$$\text{dist}_A(v) = 0$$

$$\text{dist}_A(v) = 1$$

$$\text{dist}_A(v) = 2$$

$$\text{dist}_A(v) = 3$$

Theorem 1. A graph is bipartite if and only if it does not contain any cycle of odd length.

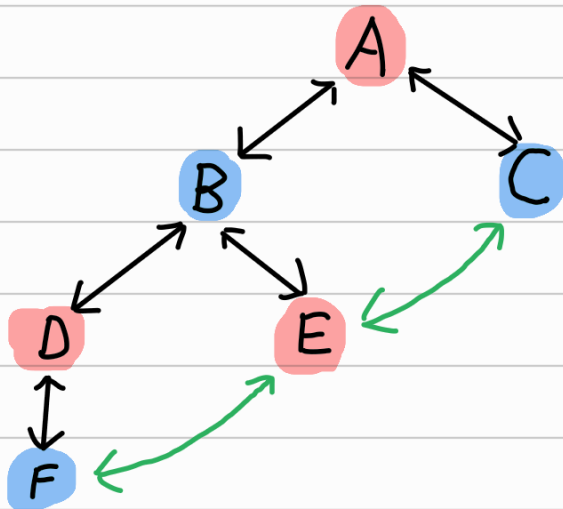
sei $v \in V$ beliebig

$V_1 = \{u \in V \mid \text{min. Distanz von } v \text{ zu } u \text{ ist } \underline{\text{ungerade}}\}$

$V_2 = \{u \in V \mid \text{min. Distanz von } v \text{ zu } u \text{ ist } \underline{\text{gerade}}\}$

V_1, V_2 bilden eine Bipartition

sei $v = A$: $V_1 = \{B, C, F\}$ $V_2 = \{A, D, E\}$



Pseudocode

```
BFS(Adj):  
    n = Adj.length  
    visited[1...n] = {false, ..., false}  
  
    for v = 1...n  
        if visited[v] == false:  
            Path(v, Adj, visited)
```

```
Path(v, Adj, visited)  
    Q = [v]  
    while Q not empty:  
        u = dequeue(Q)  
        visited[u] = true  
        for w in Adj[u]  
            if not visited[w]  
                enqueue(Q, w)
```

benutzt Queue: FiFo. First-in-First-out

$$O(|V| + |E|)$$

$$O(|E|) \quad |V|-1$$

Vergleich DFS

```
DFS(Adj):
    n = Adj.length
    visited[1...n] = {false, ..., false}

    for v = 1...n
        ✓ if visited[v] == false:
            Path(v, Adj, visited)
```

```
Path(v, Adj, visited):
    visited[v] = true
    for u in Adj[v]:
        if visited[u] == false:
            Path(u, Adj, visited)
```

```
DFS(Adj):
    n = Adj.length
    visited[1...n] = {false, ..., false}

    for v = 1...n
        if visited[v] == false:
            Path(v, Adj, visited)
```

```
✓ Path(v, Adj, visited)
    S = [v]
    while S not empty:
        u = pop(S)
        visited[u] = true
        for w in Adj[u]
            if not visited[w]
                push(S, w)
```

```

BFS(Adj):
  n = Adj.length
  visited[1...n] = {false, ..., false}

  for v = 1...n
    if visited[v] == false:
      Path(v, Adj, visited)

```

```

Path(v, Adj, visited)
  Q = [v]
  while Q not empty:
    u = dequeue(Q)
    visited[u] = true
    for w in Adj[u]
      if not visited[w]
        enqueue(Q, w)

```

```

DFS(Adj):
  n = Adj.length
  visited[1...n] = {false, ..., false}

  for v = 1...n
    if visited[v] == false:
      Path(v, Adj, visited)

```

```

~ Path(v, Adj, visited)
  S = [v]
  while S not empty:
    u = pop(S)
    visited[u] = true
    for w in Adj[u]
      if not visited[w]
        push(S, w)

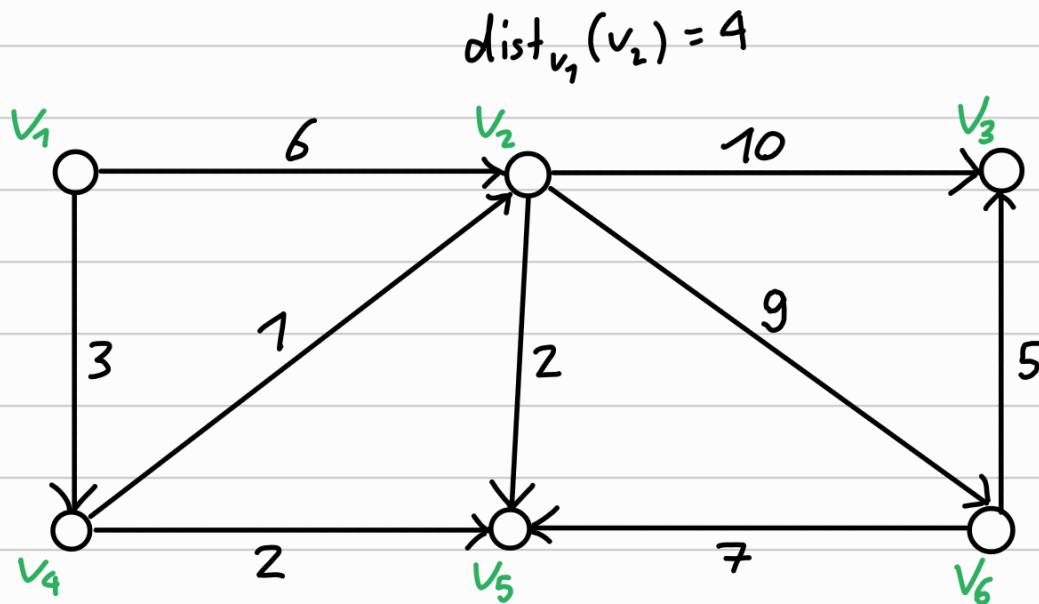
```

Laufzeit BFS / DFS : $O(|V| + |E|)$

Def. gewichteter Graph

gerichteter Graph + Gewichtsfunktion c

$$c: E \rightarrow \mathbb{R}$$



Shortest Path Problem

in gewichtetem Graph $G = (V, E)$, $c: E \rightarrow \mathbb{R}$

für $v \in V$, was ist die min. Distanz von v zu allen anderen Knoten $u \in V$

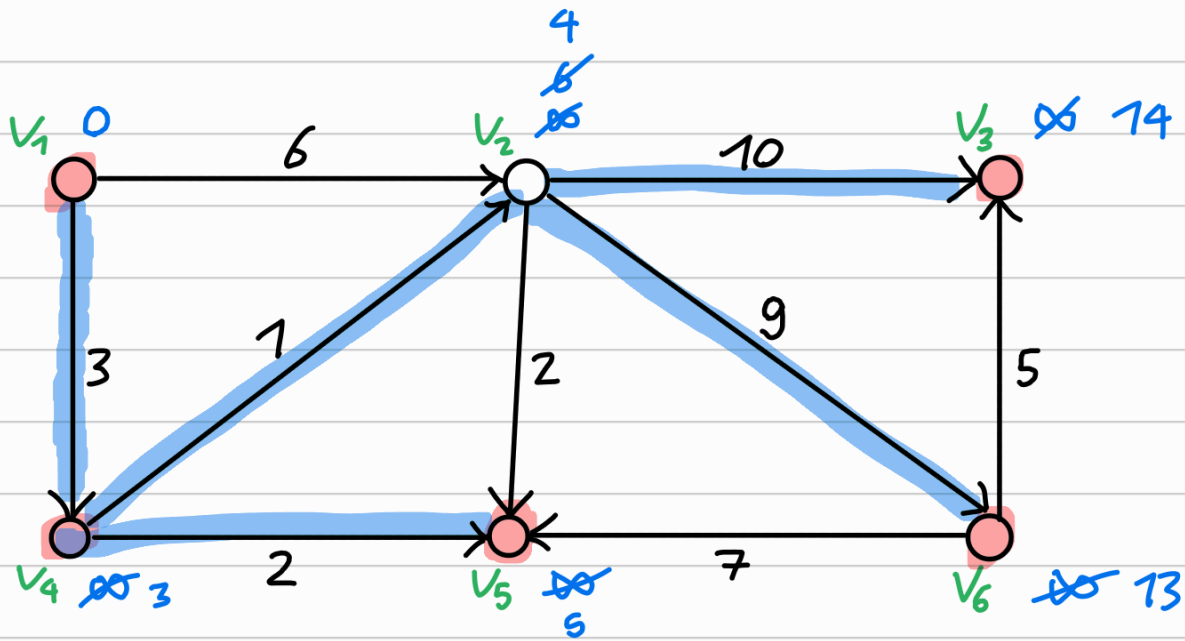
$$\text{dist}_v(u)$$

Wenn alle Gewichte nicht negativ ($\forall e \in E \ c(e) \geq 0$)

→ Dijkstra

Dijkstra

Beispiel: Dijkstra(G, v_1)



$$0 + 3 < \infty ?$$

$$\text{dist}(v_1) = 0 \quad \text{dist}(v_2) = 4 \quad \text{dist}(v_3) = 14$$

$$\text{dist}(v_4) = 3 \quad \text{dist}(v_5) = 5 \quad \text{dist}(v_6) = 13$$

Pseudocode

s: start, adj: Adjazenzliste, c: Gewichte

```
Dijkstra(adj, c, s)
  d[1...n] = {inf, ..., inf}
  d[s] = 0
  PQ = PriorityQueue( [(1, d[1]), ..., (n, d[n])] )

  while PQ not empty:
    u = PQ.poll()      oder PQ.extractMin
    // poll() -> v in PQ mit d[v] minimal

    for v in adj[u]
      if v in PQ
        d[v] = min(d[v], d[u] + c(u, v) )
        PQ.decreaseKey((v, d[v]))

  return d
```

Datenstruktur : Priority Queue

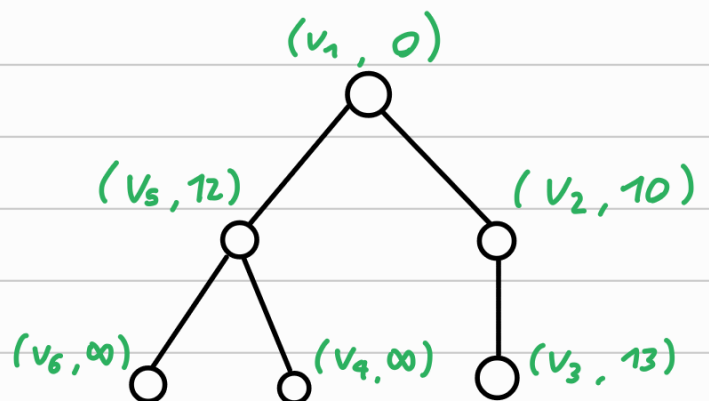
Implementiert als Min-Heap

Insert

ExtractMin / Poll

DecreaseKey

alle in $O(\log n)$



In Java

```
public static int Dijkstra(int start, ArrayList<ArrayList<Integer>> E,
    ArrayList<ArrayList<Integer>> C) {
    n = E.size();
    Tuple[] dist = new Tuple[n];
    for(int i = 0; i < n; i++){
        int startdist = Integer.MAX_VALUE/2;
        if(i == start) startdist = 0;
        dist[i] = new Tuple(i, startdist);
    }

    PriorityQueue<Tuple> PQ = new PriorityQueue<Tuple>();
    for(int i = 0; i < n; i++){
        PQ.add(dist[i]);
    }

    while(PQ.size() > 0){
        Tuple vd = PQ.poll();
        int v = vd.v;
        int d = vd.dist;

        // Alter Eintrag. Distanz wurde bereits verbessert
        if (d > dist[v].dist) continue;

        // Distanz zu Nachbarn von v updaten
        ArrayList<Integer> neighbors = E.get(v);
        ArrayList<Integer> neighborWeights = C.get(v);
        for(int i = 0; i < neighbors.size(); i++){
            int u = neighbors.get(i);
            int w = neighborWeights.get(i);

            //update
            if( d + w < dist[u].dist){
                dist[u].dist = d + w;
                PQ.add(dist[u]);
            }
        }
    }

    return dist; // dist[i].dist == Distanz von start zu i
}
```

oder $PQ.delete(dist[u])$
 $dist[u].dist = d + w$
 $PQ.add(dist[u])$
(aber schlechte Laufzeit)

```

import java.util.ArrayList;
import java.util.PriorityQueue;

static class Tuple implements Comparable<Tuple> {
    int v;
    int dist;

    public Tuple(int v, int dist) {
        this.v = v;
        this.dist = dist;
    }

    @Override public int compareTo(Tuple other) {
        return this.dist - other.dist;
    }
}

```

Laufzeit Dijkstra :

$$\begin{aligned}
 |V| \text{ Mal } PQ.\text{extractMin} &: |V| \cdot \log(|V|) \\
 &+ \\
 |E| \text{ Mal } PQ.\text{decreaseKey} &: |E| \cdot \log(|V|)
 \end{aligned}$$

→ $O((|V| + |E|) \cdot \log(|V|))$

Dijkstra Korrektheitsbeweis Dijkstra($G=(V,E)$, s)

(Annahme $\forall e \ c(e) > 0$)

Zu beweisen: ein kürzester Pfad von s zu v
nutzt keine Knoten, die nach $PQ.poll() = v$
noch in PQ sind

Base Case: erster Aufruf $PQ.poll()$ gibt s , da
 $dist_s(s) = 0$. kürzester Pfad braucht nur s .

Induktionshypothese: Betrachte PQ zu beliebigem Zeitpunkt.

Die Aussage gilt für alle $v \in V \setminus PQ$

Induktionsschritt sei $PQ.poll() = v$

$\Rightarrow \forall u \in PQ \quad dist_s(v) \leq dist_s(u) \quad (\text{def. poll})$

sei u^* der letzte Knoten vor v im kürzesten s - v Pfad

also $dist_s(u^*) + c(u^*, v) = dist_s(v)$

Annahme: u^* ist noch in PQ

da $\forall e \ c(e) > 0$ ist $\text{dist}(u^*) < \text{dist}_s(v)$

Ein Widerspruch zu $\text{PQ.poll}() = v$

u^* ist nicht mehr in PQ

$\Rightarrow$ ein kürzester $s-u^*$ Pfad nutzt keine Knoten in PQ (I.H.)

$\Rightarrow$ ein kürzester $s-v$ Pfad nutzt keine Knoten in PQ



Wir haben bewiesen:

ein kürzester Pfad von s zu v
nutzt keine Knoten, die nach $\text{PQ.poll}() = v$
noch in PQ sind

die Dijkstra Update Regel garantiert zu jedem Zeitpunkt

$$\text{dist}_s(v) = \min \{ \text{dist}_s(u) + c(u, v) \mid (u, v) \in E, u \in V \setminus \text{PQ} \}$$

Damit können wir per Induktion beweisen:

nach $\text{PQ.poll}() = v$ ist $\text{dist}_s(v)$ minimal.

Vorbesprechung Ex. Sheet 10

10.1	DFS	Bonus
10.2	BFS	Bonus
10.3	Graph Modellierung	wichtig
10.4	Dijkstra von Hand	wichtig
10.5	DFS Anwendung	Bonus

Peergrading

Ex. 9.4 b)